

Adversarial Dependent Type Theory

Luc Alexander Lüthi

April 6th, 2026

Contents

1	Introduction	1
2	Implication as an Object	1
3	Constraint	2
4	Malware as Propaganda	2

1 Introduction

Previous models of claim based adversarial security modeling relied largely on a Curry-Howard esque relation between arguments and programs. Arguments are programs, claims are instructions, and evidence is an input for proof as an output. What previous representations lacked, and why they failed to form decent substrates for adversarial modeling, was constraint fulfillment. When you write a program, you can call it anything you want, claim it does anything you want, and implement it however you want. What actually determines whether or not you have actually completed the task at hand or just faked it? The answer is constraints, in programming usually external observers of the output which determine whether or not it is correct. In type theory we can emulate these observers to provide more constraint and rigor to the construction of our programs and provide higher assurances that the program we just wrote actually accomplishes what we claim it accomplishes. Arguments can follow this same logic. By making proofs inspectable, and allowing our proofs to be constrained by what dependencies they have fulfilled allows our arguments to encode whether or not it has actually been articulately completed or if its lacking substance or evidence. Dependency in argument is a constraint on evidence.

In this note I present an adversarial model of dependent type theory as applied to claims and the arguments they encode. I then show how this can be applied to security modeling to demonstrate examples of malware and exploitation from the perspective of arguments and propaganda.

2 Implication as an Object

The base unit of our representation is an implication. $A \rightarrow B$, which encodes that A is evidence of a proof of B . An implication can be thought of as a parametric constructor of a proof taking two arguments. Implications can be named, or colored. Named implications may have bodies the way functions may have bodies in computation. These constitute how the implication encodes the proof it claims, and may both invoke and fulfil constraints at different times. There are two notations for implication, both named and otherwise. Either prefix:

$\text{Impl } a \ b$

$F \ a \ b$

Or infix with named arrows, we will procede with this method for the remainder of the note, but my preference has not been determined as of yet and future notes may use the prefix "instruction" style as opposed to the infix "theoretic" style:

$$a \rightarrow b$$

$$a \rightsquigarrow b$$

In either notation lowercase characters are used to denote parameters to the constructor of the proof, and uppercase characters are used to denote defined state. To define the body of a named implication we denote sequential claims followed by a conclusion.

$$a \rightsquigarrow b :$$

$$a \rightarrow C$$

$$C \rightarrow b$$

$$a$$

$$a \rightarrow b$$

Any implication can be inspected to encode dependency and for ce constraint. A dependency can be thought of as an obligation which must be proven as a subgoal in order for the proof to succeed. This is denoted as the proof containing a subproof:

$$a \rightarrow ((a \rightarrow C) \in b)$$

3 Constraint

In order for a proof to succeed given evidence, all body claims must resolve to true. Each subclaim to an implication can be thought of as a constraint on the arguments of the implication. There are two kinds of existential constraints which can be leveraged in a body, dependency constraints and proof obligation constraints. A proof obligation constraint is a simple claim that a proof exists, whether on its own or with specified evidence. These are made with any implication. A dependency constraint can be made by asserting a dependency of an otherwise obligatory constraint.

$$a \rightsquigarrow b :$$

$$(b \rightarrow C) \in a$$

$$a \rightarrow C$$

$$C \rightarrow b$$

$$a$$

$$a \rightarrow b$$

4 Malware as Propaganda

When encoding security as claims and arguments, we must consider what exploitation looks like in this analogy. Indeed named implication are functions, but where does the analogy bring us when we misuse those functions. Adversarial argumentation looks a lot like convincing someone of something which benefits you, this is done through a variety of techniques, all requiring a specified environment.

Our environment will consist of a universe of minds. Minds can contains proofs. Minds gain new proofs by being exposed to signals in the environment. Minds may construct new proofs using inherent proofs, and construct signals using inherent proofs to send to other minds in the universe. Each mind has a

set of capability states which when proven provide the mind external abilities. These depend on the staging of the environment but may correspond well to physical movement, cognition, signaling itself, proof construction itself, etc. It is in the minds best interest to maintain stable unexploitable proofs of these capability states while preventing proofs of other not so beneficial states. It is important that minds cannot know by default where any given signal came from, although depending on the representation of the environment this may become obvious in some instantiations.

A game can now be played where each mind creates defensive proofs of its useful capabilities while constraining the proofs to ensure they do not allow harmful capabilities to be proven, while attackers attempt to send signals with evidence sufficient to instantiate proofs of harmful capabilities. Attackers may send signals which have ulterior motives and make it easier to exploit a proof for gaining a certain capability, or may send signals to fulfil constraints in an opposing mind before sending a final payload. This is argumentation as exploitation, which is exactly propaganda. Malware takes the form of harmful ideas, parasitism manifests as a useful signal for proving a usefui capability with insufficient constraints for preventing adversarial gain.